

Kubernetes Storage

The Complete Masterclass

CSI · Dynamic Provisioning · AWS EBS · StatefulSets · Production Architecture

8 Phases — Beginner to Production-Grade

Phase 1: Core Foundations | Phase 2: PV/PVC Deep Dive | Phase 3: CSI Architecture

Phase 4: AWS Storage | Phase 5: Production Scenarios | Phase 6: Advanced Concepts

Phase 7: Practical Labs | Phase 8: Interview Mastery

Phase 1 — Core Foundations

Before we dive into YAML manifests, controllers, and drivers, we need to build a solid mental model. The best engineers are not those who memorize commands — they are those who understand WHY things exist. Every component of Kubernetes storage exists to solve a specific, painful problem. Understanding those problems first makes everything else click.

1.1 Why Do Containers Need Persistent Storage?

Let us start with the most fundamental question. A container is, at its heart, an isolated process running on a Linux kernel. Everything inside the container — its filesystem, its files, its database state — lives in a layered filesystem that is created when the container starts and destroyed when the container stops.

This is intentional. The ephemeral nature of containers is what makes them fast to start, easy to reproduce, and simple to distribute. The problem is that not all applications are stateless. A PostgreSQL database needs to store data. A Kafka broker needs to persist message logs. An Elasticsearch node needs to maintain its index files.

When the pod running PostgreSQL crashes, Kubernetes restarts it on the same or a different node. The new pod starts from the container image — a clean slate. Without persistent storage, all your database records are gone. Every application restart is a data wipe. This is catastrophic.

Persistent volumes solve this by decoupling the storage lifecycle from the pod lifecycle. The volume exists independently of any pod. Pods are born and die; the storage persists.

1.2 The Intuitive Mental Model — Analogies First

Think of it as renting an office

Imagine you are a startup company (your application). You need office space (storage) to run your business. Here is how the Kubernetes storage analogy works:

Kubernetes Concept	Real-World Analogy
PersistentVolume (PV)	An actual physical office space in a building. It exists whether or not someone is using it. It has a specific size, location, and key.
PersistentVolumeClaim (PVC)	A lease request form your company submits: 'I need 100 sq ft of office space in Mumbai.' The leasing office matches you with an available space.
StorageClass	The leasing policy or tier: 'Economy offices' (slow HDD), 'Business offices' (standard SSD), 'Premium offices' (ultra-fast NVMe). Each class has different prices, speeds, and features.
CSI Driver	The building management company that actually owns and operates the offices. Kubernetes (the tenant) communicates with them through a standard contract (CSI specification) to provision, resize, or decommission offices.

Kubernetes Concept	Real-World Analogy
Pod	Your employee who actually uses the office. If an employee leaves (pod dies), the office is not demolished — it stays ready for the next employee (new pod).

What happens when a pod dies — the storage survival story

Here is the sequence of events that shows why storage persistence matters:

1. Day 1: Your PostgreSQL pod starts. It is given access to a 50GB volume. It initializes its data directory, creates tables, and begins writing millions of rows.
2. Day 3: A node failure occurs. Kubernetes detects the pod is gone. It reschedules a new PostgreSQL pod on another healthy node.
3. Day 3 (seconds later): The new pod starts, gets attached to the SAME 50GB volume, finds all 3 days of data intact, and resumes operations.
4. Day 3 (outcome without PV): The new pod would start fresh with an empty container filesystem. All data is gone. Your application is broken.

The PersistentVolume is the physical storage. It is NOT the pod. It is NOT the container. It is the persistent entity that outlives everything else.

1.3 Storage Types — From Ephemeral to Production-Grade

Storage Type	Characteristics, Use Cases, and Limitations
emptyDir	Created when a pod starts, deleted when pod dies. Shared between containers in the SAME pod. Use for: temporary scratch space, caching, inter-container file sharing. Never for: data that must survive pod restart. Lives on the node's local disk or RAM (if medium: Memory).
hostPath	Mounts a directory from the node's filesystem into the pod. Survives pod restarts IF pod lands on the same node. Dangerous in production: pods can read sensitive host files, security risk, not portable across nodes. Use for: node-level monitoring agents (Prometheus node-exporter), debugging only.
Local PV	A more formalized hostPath. Explicitly tied to a specific node using node affinity. Kubernetes scheduler ensures pods using this PV always land on the correct node. Use for: single-node databases in dev/test, or high-performance workloads where network latency is unacceptable. Limitation: if the node dies, data is gone.
NFS (Network File System)	A network-attached filesystem. Multiple pods can read and write simultaneously (RWX access mode). Use for: shared configuration files, shared media storage, legacy application

Storage Type	Characteristics, Use Cases, and Limitations
	shared storage. Limitation: NFS server is a single point of failure, network latency, not ideal for high-IOPS databases.
Cloud Block Storage (EBS, GCE PD, Azure Disk)	A network-attached block device provided by the cloud. One volume attached to one node at a time (RWO). High performance, durability, automatic replication within an AZ. Ideal for: databases, message queues, any stateful workload needing durability. Limitation: must be in the same AZ as the node.
Cloud File Storage (EFS, Azure Files)	Managed NFS-compatible storage from the cloud. Multiple nodes/pods can access simultaneously (RWX). Elastic scale, no capacity management. Use for: shared file systems, content management, ML training data. Higher latency than block storage.
Object Storage (S3, GCS)	Not a filesystem — accessed via HTTP API. Not mountable as a standard volume in Kubernetes natively. Infinite scale, cheapest storage. Use for: application data assets, backups, log archives, ML model storage. Access via SDK or FUSE-based mounts (with limitations).

1.4 The Ephemeral Storage Problem in Production

Let us walk through a real production failure scenario to understand exactly why ephemeral storage is dangerous:

Scenario: A team runs a Redis cache cluster in Kubernetes using only emptyDir volumes. Redis is configured to periodically snapshot its in-memory data to disk (RDB persistence). Everything runs fine for weeks. Then:

5. A memory pressure event causes the Kubernetes node to be evicted and restarted
6. The Redis pods are terminated — emptyDir volumes are deleted
7. New Redis pods start on new nodes — empty data directories
8. Redis starts serving empty cache responses
9. The application cannot distinguish 'cache miss' from 'data is gone'
10. The backend database receives 10x traffic suddenly — it cannot handle the load
11. The database becomes the bottleneck — the entire system degrades

This is called a cache stampede triggered by ephemeral storage loss. The fix: PersistentVolumes backed by cloud block storage. Even if the pod is rescheduled to a different node, the EBS volume is detached from the old node and attached to the new node — Redis finds its RDB snapshot and warm-starts.

1.5 Interview Questions — Phase 1

Q: What is the difference between emptyDir and a PersistentVolume?

A: emptyDir is created and destroyed with the pod — it is pod-scoped storage. A PV exists independently of any pod and survives pod termination, rescheduling, and even node failures (for cloud-backed storage). emptyDir is for temporary scratch space; PVs are for data that must persist.

Q: Can a hostPath volume be used in production on a multi-node cluster? Why or why not?

A: hostPath is problematic in production multi-node clusters because it ties the application to a specific node. If Kubernetes schedules the pod on a different node, it sees an empty (or wrong) directory. It also creates security risks — a misconfigured hostPath can expose sensitive host filesystem paths to containers. Use Local PVs with explicit node affinity if you need node-local storage, and only when performance requirements justify the lack of data durability.

Q: What is the fundamental problem that PersistentVolumes solve?

A: PersistentVolumes decouple the storage lifecycle from the pod lifecycle. Containers are ephemeral by design — their local filesystem is destroyed when they terminate. For stateful applications (databases, message queues, file servers), data must outlive any individual container or pod. PVs provide a mechanism for Kubernetes to manage external, durable storage resources independently of the workloads that consume them.

Phase 2 — PV/PVC Deep Understanding

Now that you have the mental model, let us go deep into how PersistentVolumes and PersistentVolumeClaims actually work inside Kubernetes. This section builds from the binding process all the way to reclaim policies and access modes.

2.1 Static Provisioning — The Manual Approach

In static provisioning, a cluster administrator pre-creates PersistentVolumes that represent actual storage resources. Developers then claim those volumes via PersistentVolumeClaims. Think of an admin manually setting up office space before anyone requests it.

Static PV Example — AWS EBS Manual

```
# Administrator creates the PV manually
apiVersion: v1
kind: PersistentVolume
metadata:
  name: postgres-pv-az1a
  labels:
    app: postgres
    env: production
spec:
  capacity:
    storage: 100Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  awsElasticBlockStore:
    volumeID: vol-0a1b2c3d4e5f67890
    fsType: ext4
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: topology.kubernetes.io/zone
              operator: In
              values:
                - ap-south-1a
```

```
# Developer creates PVC to claim the volume
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
  namespace: production
spec:
  accessModes:
    - ReadWriteOnce
```

```
storageClassName: manual
resources:
  requests:
    storage: 100Gi
```

Static provisioning limitations: The admin must know in advance how much storage each team needs. If a team needs 150Gi but the pre-created PV is 100Gi, it cannot bind (PVC requests more than PV offers). Storage is often wasted — a 100Gi PV claimed by a 10Gi PVC still allocates the full 100Gi. This is why dynamic provisioning was invented.

2.2 Dynamic Provisioning — The Automatic Approach

Dynamic provisioning is the modern, production standard approach. Instead of an admin pre-creating PVs, a StorageClass defines a template and a provisioner. When a PVC is created that references a StorageClass, the provisioner automatically creates a PV (and the underlying cloud resource — like an EBS volume) on demand.

This is like a hotel. Instead of pre-allocating specific rooms to specific guests, you just specify 'I need a Standard room for 3 nights.' The hotel system allocates the best available room automatically when you arrive.

StorageClass Definition

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
  encrypted: "true"
  kmsKeyId: arn:aws:kms:ap-south-1:123456:key/abc-def
reclaimPolicy: Delete
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
allowedTopologies:
- matchLabelExpressions:
  - key: topology.kubernetes.io/zone
    values:
      - ap-south-1a
      - ap-south-1b
```

Each field matters:

- **provisioner** — which CSI driver handles this class. `ebs.csi.aws.com` is the AWS EBS CSI driver.

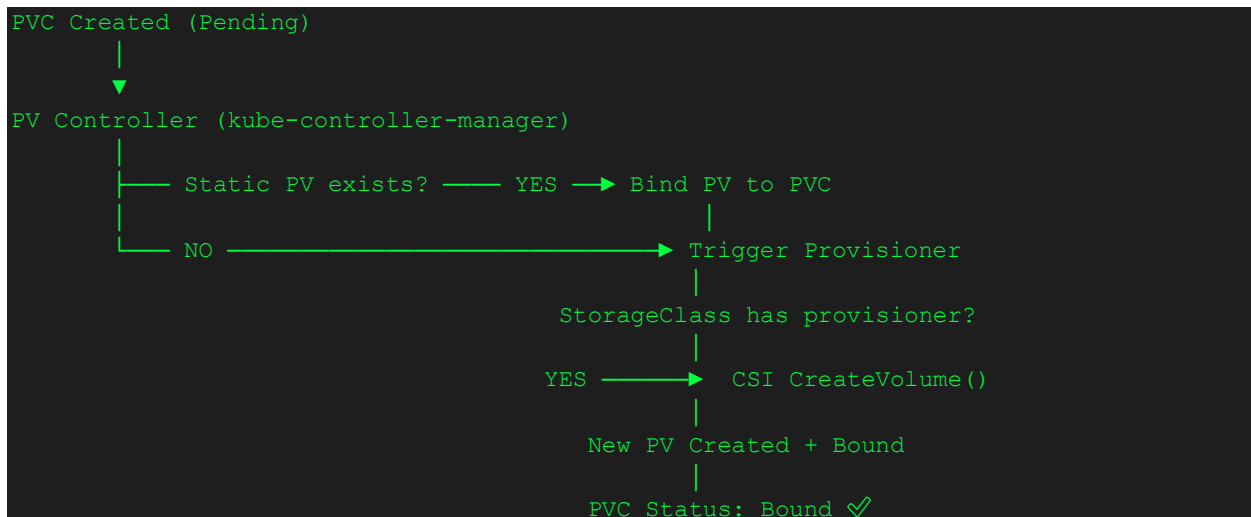
- parameters — driver-specific settings. These are passed directly to the cloud API.
- reclaimPolicy: Delete — when the PVC is deleted, the EBS volume is also deleted. reclaimPolicy: Retain keeps the volume.
- allowVolumeExpansion: true — enables resizing volumes without recreation.
- volumeBindingMode: WaitForFirstConsumer — critical for multi-AZ clusters. Explained in Phase 6.

2.3 The Binding Process — Step by Step

Understanding binding is critical for debugging storage issues. Here is the exact sequence of events when a PVC is created:

12. PVC is created — Kubernetes API server stores the PVC object with status: Pending
13. kube-controller-manager's PV controller watches for new PVCs
14. PV controller searches for a matching PV: storage class matches, access mode matches, capacity is sufficient (PV capacity $\geq$ PVC request), labels match (if selector is specified)
15. If a matching static PV is found: PV controller binds them by setting pv.spec.claimRef to point to the PVC and pv.status.phase to Bound. PVC status changes to Bound.
16. If no matching PV exists and StorageClass has a provisioner: PV controller triggers the external-provisioner sidecar which calls the CSI driver to create a new volume.
17. CSI driver creates the cloud volume (e.g., AWS API call: CreateVolume) and returns the volume ID
18. A new PV object is created with volumeHandle set to the cloud volume ID
19. PV and PVC are bound together
20. Scheduler sees the PVC is Bound and can now schedule pods that reference it

Binding Diagram



2.4 Access Modes — Who Can Read and Write

Access Mode	Description and Practical Meaning
ReadWriteOnce (RWO)	The volume can be mounted read-write by ONLY ONE NODE at a time. Multiple pods on the SAME node can use an RWO volume. This is the mode for AWS EBS — a volume can only be attached to one EC2 instance. Use for: databases, single-instance applications.
ReadOnlyMany (ROX)	The volume can be mounted read-only by many nodes simultaneously. Use for: serving static assets, configuration files, read-only reference data.
ReadWriteMany (RWX)	The volume can be mounted read-write by many nodes simultaneously. AWS EBS does NOT support this. Use EFS (NFS-based) for RWX. Use for: shared content platforms, collaborative applications, ML training data.
ReadWriteOncePod (RWOP)	NEW in Kubernetes 1.22. Like RWO but restricted to a SINGLE POD (not just a single node). Ensures truly exclusive access even if multiple pods are on the same node. Supported by CSI drivers that implement SINGLE_NODE_SINGLE_WRITER capability.

❑ **A common mistake: Trying to use an EBS volume with ReadWriteMany. EBS is a block device that can only be attached to one EC2 instance at a time. If you need RWX on AWS, use EFS (with the EFS CSI driver) or FSx for Lustre.**

2.5 Reclaim Policies — What Happens When You Delete a PVC

Reclaim Policy	Behavior and When to Use
Delete	When the PVC is deleted, the PV is deleted, AND the underlying cloud storage (EBS volume) is deleted. This is the default for dynamically provisioned volumes. Data is permanently lost. Use for: stateless-ish workloads, dev/test environments, any case where data is recreatable.
Retain	When the PVC is deleted, the PV remains with status Released. The underlying cloud storage is NOT deleted. The data is safe but the PV cannot be bound to a new PVC until manually reclaimed. An admin must manually inspect the data, create a new PV, and bind. Use for: production databases where data loss is unacceptable.
Recycle	DEPRECATED. Performed a basic <code>rm -rf</code> on the volume and made it available again. Removed because it was unsafe and limited. Do not use.

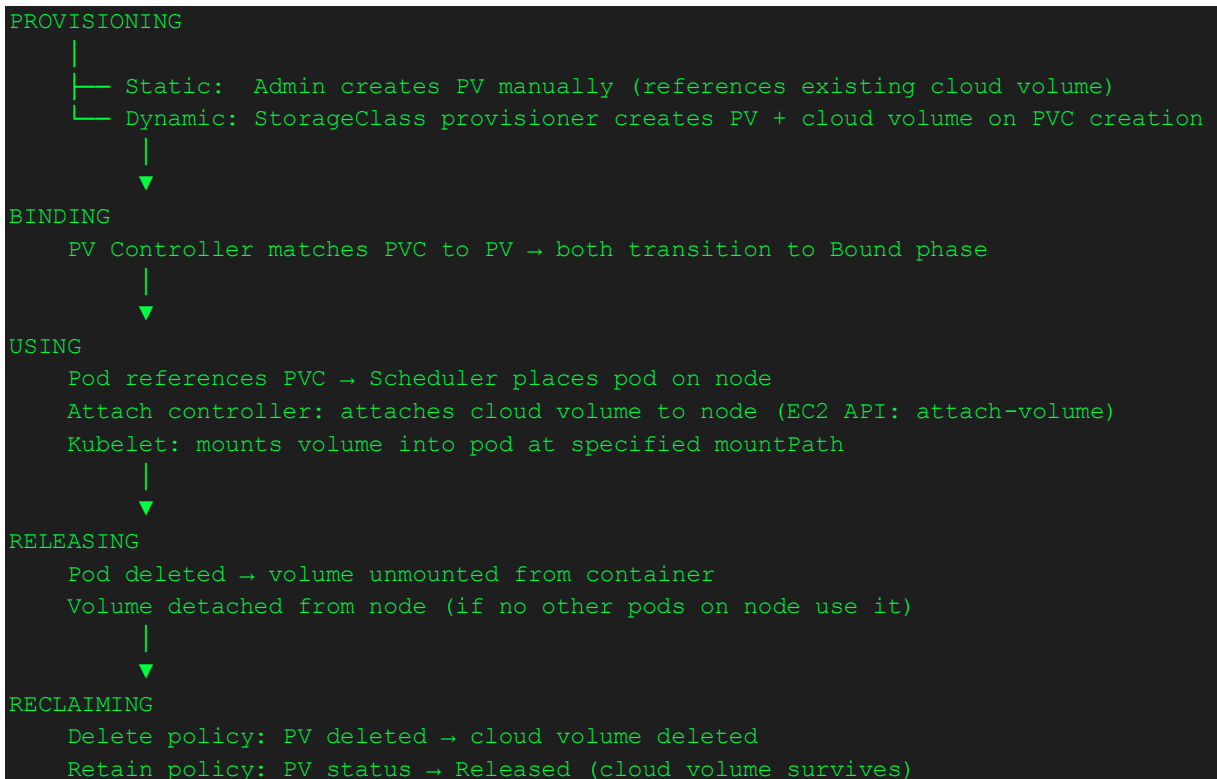
Production pattern: Retain + GitOps

In production, set reclaimPolicy: Retain on database StorageClasses. When a StatefulSet is deleted (intentionally or accidentally), the EBS volumes survive. You can then create a new StatefulSet and manually bind the old PVs to new PVCs to restore from the existing data.

2.6 Volume Modes — Filesystem vs Block

Volume Mode	Description
Filesystem (default)	Kubernetes formats the block device with a filesystem (ext4, xfs) and mounts it as a directory inside the container. The application reads and writes files normally. 99% of use cases.
Block	The raw block device is presented to the container without formatting. The application or database engine manages the device directly. Required for some high-performance database deployments (Oracle, some PostgreSQL configurations) that do their own I/O scheduling. Extremely rare in typical Kubernetes workloads.

2.7 The Volume Lifecycle — Full Picture



2.8 Interview Questions — Phase 2

Q: A PVC is stuck in Pending state. What are the possible causes?

A: (1) No matching PV exists and no StorageClass provisioner is configured. (2) StorageClass provisioner is failing — check CSI driver logs. (3) Resource quota exceeded in the namespace. (4) Access mode mismatch — PVC requests RWX but only RWO PVs exist. (5) Storage capacity too high — no PV with sufficient capacity exists. (6) volumeBindingMode: WaitForFirstConsumer — no pod has been scheduled yet to determine which AZ to provision in.

Q: What is the difference between reclaimPolicy: Delete and reclaimPolicy: Retain?

A: Delete means when the PVC is deleted, both the PV object and the underlying cloud volume are deleted — data is permanently lost. Retain means the PV object and cloud volume both survive PVC deletion, with PV status changing to Released. The data is safe but requires manual admin intervention to rebind. In production, databases should use Retain to prevent accidental data loss.

Q: Can two pods on different nodes share an EBS volume?

A: No. EBS volumes support ReadWriteOnce — they can only be attached to a single EC2 instance at a time. Two pods on DIFFERENT nodes cannot share the same EBS volume simultaneously. However, two pods on the SAME node can both mount the same RWO volume. For multi-node shared storage on AWS, use EFS (ReadWriteMany support) with the EFS CSI driver.

Phase 3 — CSI Architecture

This is the most technically deep section of the document. Understanding CSI (Container Storage Interface) is what separates junior Kubernetes engineers from senior platform engineers. CSI is the plumbing that connects Kubernetes to cloud storage APIs, and understanding it lets you debug any storage issue at any level.

3.1 Why CSI Was Introduced — The Problem Before CSI

Before CSI (pre-Kubernetes 1.13), storage drivers were built directly into the Kubernetes core codebase. This was called 'in-tree' storage. Every cloud provider had to submit code directly to the Kubernetes repository to add storage support.

The problems with in-tree storage were severe:

- Bug fixes in storage drivers required a Kubernetes release — a months-long process
- Storage vendors had no control over their own driver's release schedule
- The Kubernetes codebase grew bloated with dozens of storage implementations
- New storage systems could not be added without modifying Kubernetes source code
- Testing was difficult — all storage code was in the same binary

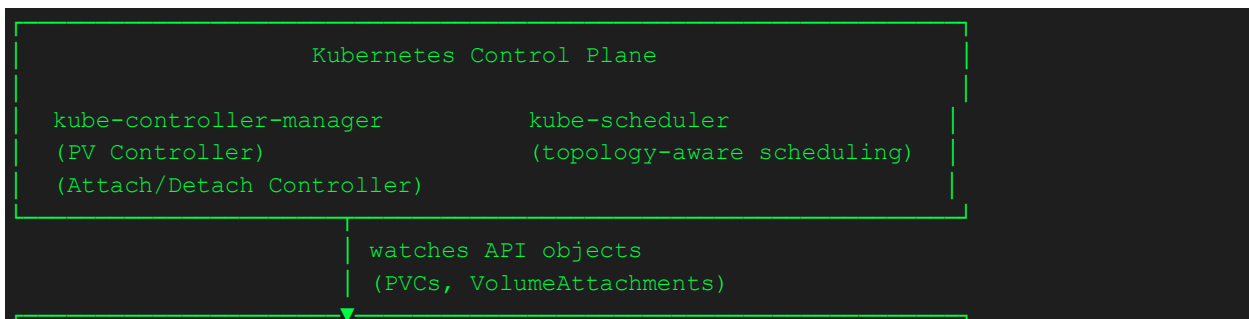
The in-tree AWS EBS driver in Kubernetes literally contained code that made direct AWS API calls from inside the Kubernetes binary. If AWS changed an API, Kubernetes had to release a new version.

3.2 CSI — The Solution

CSI (Container Storage Interface) is a standardized API specification that defines how container orchestrators (Kubernetes) communicate with storage systems. Think of CSI as an electrical outlet standard — any device that follows the standard can plug into any wall socket. Any storage vendor that implements the CSI spec can work with any CSI-compatible orchestrator.

With CSI, storage drivers run as separate pods in the Kubernetes cluster — NOT inside the Kubernetes core binaries. They are called 'out-of-tree' drivers. Storage vendors can release updates independently, on their own schedule, without waiting for a Kubernetes release.

3.3 CSI Architecture — Components Deep Dive





3.4 CSI Sidecar Components Explained

Sidecar	Function and Responsibility
external-provisioner	Watches the Kubernetes API for new PVCs that reference a StorageClass with this driver's provisioner name. When found, calls the CSI driver's CreateVolume gRPC method. The driver creates the cloud volume (e.g., AWS EBS CreateVolume API call) and returns a volume handle. external-provisioner then creates a PV object in Kubernetes.
external-attacher	Watches VolumeAttachment objects created by kube-controller-manager. When a VolumeAttachment appears (meaning a pod needs a volume attached to a node), calls ControllerPublishVolume gRPC. For EBS, this triggers the AWS EC2 AttachVolume API call, attaching the EBS volume to the EC2 instance.
external-resizer	Watches PVCs for capacity increase requests. When a PVC's spec.resources.requests.storage is increased, calls

Sidecar	Function and Responsibility
	ControllerExpandVolume gRPC. For EBS, this calls AWS ModifyVolume to increase the disk size. Then calls NodeExpandVolume on the node to resize the filesystem.
external-snapshotter	Watches VolumeSnapshot objects. When a snapshot is requested, calls CreateSnapshot gRPC. For EBS, creates an AWS EBS snapshot.
node-driver-registrar	Runs on every node as part of the DaemonSet. Registers the CSI driver with the kubelet using the kubelet plugin registration mechanism. After registration, kubelet knows how to forward mount/unmount requests to the CSI driver.
liveness-probe	Monitors the CSI driver's health by calling Probe() gRPC. Reports unhealthy status to Kubernetes if the driver is not responding, allowing pod restart.

3.5 The Complete Dynamic Provisioning Flow — EBS Example

Let us trace exactly what happens when you create a PVC that requests a new EBS volume. Every line of code that runs, every API call that is made:

21. You apply a PVC YAML with storageClassName: ebs-gp3 and storage: 50Gi
22. Kubernetes API server stores the PVC object. Status: Pending
23. external-provisioner pod (running in kube-system or driver namespace) detects the new PVC via its watch on the API server
24. external-provisioner checks: Is there an existing PV that matches? No. Does the StorageClass have this driver as provisioner? Yes.
25. external-provisioner calls CreateVolume gRPC on the CSI driver container via Unix socket
26. CSI driver container calls AWS EC2 API: `aws ec2 create-volume --size 50 --volume-type gp3 --availability-zone ap-south-1a`
27. AWS API responds with volume ID: `vol-0abc123def456789`
28. CSI driver returns CreateVolumeResponse to external-provisioner with `volume_id=vol-0abc123def456789`
29. external-provisioner creates a PV object in Kubernetes with `spec.csi.volumeHandle=vol-0abc123def456789`
30. PV Controller sees the new PV and binds it to the PVC. Both transition to Bound phase
31. Pod referencing the PVC is scheduled to a node (e.g., `ip-10-0-1-100` in `ap-south-1a`)
32. kube-controller-manager's AttachDetach controller creates a VolumeAttachment object
33. external-attacher detects the VolumeAttachment and calls ControllerPublishVolume gRPC
34. CSI driver calls AWS EC2 API: `aws ec2 attach-volume --volume-id vol-0abc123def456789 --instance-id i-0abc123 --device /dev/xvdba`
35. AWS attaches the EBS volume to the EC2 instance. State: in-use
36. kubelet on the node detects the volume is attached via the node-driver-registrar callback

37. kubelet calls NodeStageVolume on the CSI node plugin — this formats the device with ext4 and mounts it to a staging path
38. kubelet calls NodePublishVolume — bind-mounts the staging path to the pod's specific mountPath
39. Pod starts. Container sees /data (or whatever mountPath) with 50GB available

□ *This entire flow — from PVC creation to pod startup — typically takes 20-60 seconds for EBS volumes. The AWS API calls are the slowest part.*

3.6 Interview Questions — Phase 3

Q: What is the difference between in-tree and out-of-tree CSI drivers?

A: In-tree drivers were built into the Kubernetes core binary itself, requiring Kubernetes releases for any driver update. Out-of-tree CSI drivers run as separate pods in the cluster, using a standardized gRPC API (CSI spec) to communicate with Kubernetes via sidecars like external-provisioner and external-attacher. CSI drivers can be updated independently of Kubernetes, enabling storage vendors to ship fixes and features on their own schedule.

Q: What does external-provisioner do and how is it different from the CSI driver itself?

A: external-provisioner is a sidecar that bridges Kubernetes API objects (PVCs) to CSI gRPC calls (CreateVolume). It watches for PVCs, determines if dynamic provisioning is needed, and makes the CreateVolume gRPC call to the CSI driver. The CSI driver itself has no knowledge of Kubernetes objects — it only speaks gRPC. The sidecar handles the Kubernetes-side coordination; the driver handles the storage-provider-side API calls.

Q: What triggers a volume to be attached to a node?

A: The kube-controller-manager's AttachDetach controller creates a VolumeAttachment Kubernetes object when it detects that a pod referencing a PVC has been scheduled to a node. The external-attacher sidecar watches for VolumeAttachment objects and calls ControllerPublishVolume on the CSI driver, which makes the cloud API call (e.g., AWS ec2:AttachVolume).

Phase 4 — AWS Storage Integration

This phase focuses specifically on how to use AWS storage services with Kubernetes EKS. This is where the architecture meets real cloud infrastructure.

4.1 AWS Storage Types for Kubernetes

AWS Service	Kubernetes Use Cases and Characteristics
Amazon EBS (Elastic Block Store)	Block storage. ReadWriteOnce. Same AZ as EC2 node required. Use for: databases (PostgreSQL, MySQL, MongoDB), message queues (Kafka). gp3 is the current recommended type. Max size: 64TB. Lifecycle tied to AZ.
Amazon EFS (Elastic File System)	Managed NFS. ReadWriteMany. Cross-AZ access. Elastic — no capacity to pre-provision. Use for: shared content, web serving, ML training datasets, shared configuration. Higher latency than EBS (~1-10ms vs <1ms).
Amazon FSx for Lustre	High-performance parallel filesystem. Designed for ML/HPC workloads. Can be linked to S3. Extremely high throughput and IOPS. ReadWriteMany. Use for: HPC workloads, distributed training, financial modeling.
Amazon FSx for NetApp ONTAP	Enterprise NAS/SAN. Supports NFS, SMB, iSCSI. Use for: enterprise applications migrating from on-premises NAS storage, Windows workloads.
Amazon S3	Object storage. NOT natively mountable as a Kubernetes volume. Access via AWS SDK in application code, or via S3 FUSE mounts (with limitations). Use for: backups, static assets, ML data lakes, archival.

4.2 EBS Volume Types — Performance Deep Dive

EBS Type	Use Case	Performance
gp2 (General Purpose SSD)	General purpose — legacy, being replaced by gp3	Baseline 3 IOPS/GB, bursts to 3000. Max 16,000 IOPS. Volume size determines IOPS.
gp3 (General Purpose SSD)	RECOMMENDED for most workloads	Flat 3,000 IOPS and 125 MB/s baseline regardless of size. Scale IOPS to 16,000 and throughput to 1,000 MB/s independently. 20% cheaper than gp2.
io1 (Provisioned IOPS SSD)	Legacy high-performance databases	Up to 64,000 IOPS. IOPS:GB ratio max 50:1.

EBS Type	Use Case	Performance
io2 (Provisioned IOPS SSD)	Mission-critical databases, Oracle RAC	Up to 64,000 IOPS on most instances, 256,000 on Nitro. 99.999% durability SLA. IOPS:GB ratio max 500:1.
st1 (Throughput Optimized HDD)	Log processing, data warehousing, sequential reads	Max 500 MB/s throughput, 500 IOPS. Low cost. Cannot be boot volume.
sc1 (Cold HDD)	Infrequently accessed archives	Max 250 MB/s. Lowest cost block storage option.

4.3 AWS EBS CSI Driver Installation

Prerequisites

- EKS cluster running Kubernetes 1.23+
- OIDC provider associated with the cluster
- IAM permissions for the driver to call EC2 APIs

Step 1: Create OIDC Provider

```
# Get OIDC issuer URL
aws eks describe-cluster --name tenantops-cluster \
  --query "cluster.identity.oidc.issuer" --output text

# Create OIDC provider
eksctl utils associate-iam-oidc-provider \
  --cluster tenantops-cluster --approve
```

Step 2: Create IAM Role for EBS CSI Driver (IRSA)

```
eksctl create iamserviceaccount \
  --name ebs-csi-controller-sa \
  --namespace kube-system \
  --cluster tenantops-cluster \
  --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy \
  --approve \
  --role-only \
  --role-name AmazonEKS_EBS_CSI_DriverRole
```

Step 3: Install EBS CSI Driver as EKS Addon

```
aws eks create-addon \
  --cluster-name tenantops-cluster \
  --addon-name aws-ebs-csi-driver \
  --service-account-role-arn \
    arn:aws:iam::051556733177:role/AmazonEKS_EBS_CSI_DriverRole
```

Alternative: Install via Helm

```
helm repo add aws-ebs-csi-driver \
  https://kubernetes-sigs.github.io/aws-ebs-csi-driver
helm repo update

helm upgrade --install aws-ebs-csi-driver \
  aws-ebs-csi-driver/aws-ebs-csi-driver \
  --namespace kube-system \
  --set controller.serviceAccount.create=true \
  --set controller.serviceAccount.annotations.\
    eks\.amazonaws\.com/role-
arn=arn:aws:iam::051556733177:role/AmazonEKS_EBS_CSI_DriverRole
```

Verify installation

```
kubect1 get pods -n kube-system | grep ebs-csi
# Expected output:
# ebs-csi-controller-7c9d8b7f5-abc12    6/6    Running    0    5m
# ebs-csi-controller-7c9d8b7f5-def34    6/6    Running    0    5m
# ebs-csi-node-x4rh6                    3/3    Running    0    5m
# ebs-csi-node-mdwfp                    3/3    Running    0    5m
```

4.4 Production StorageClass Configuration

```
# Production gp3 StorageClass with encryption
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-gp3-encrypted
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "3000"           # baseline, can scale to 16000
  throughput: "125"      # MB/s baseline, can scale to 1000
  encrypted: "true"
  kmsKeyId: "arn:aws:kms:ap-south-1:051556733177:key/mrk-abc123"
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Retain
allowVolumeExpansion: true
allowedTopologies:
- matchLabelExpressions:
  - key: topology.kubernetes.io/zone
    values:
      - ap-south-1a
      - ap-south-1b
```

4.5 StatefulSet with EBS — Complete Production Example

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgresql
  namespace: production
spec:
  serviceName: postgresql-headless
  replicas: 1
  selector:
    matchLabels:
      app: postgresql
  template:
    metadata:
      labels:
        app: postgresql
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: postgresql
          image: postgres:15-alpine
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              value: tenantops
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: postgres-credentials
                  key: username
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-credentials
                  key: password
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          resources:
            requests:
              memory: "512Mi"
              cpu: "250m"
            limits:
              memory: "2Gi"
              cpu: "1000m"
          volumeMounts:
            - name: postgres-data
              mountPath: /var/lib/postgresql/data
          livenessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres"]
            initialDelaySeconds: 30
```

```
    periodSeconds: 10
  readinessProbe:
    exec:
      command: ["pg_isready", "-U", "postgres"]
    initialDelaySeconds: 5
    periodSeconds: 5
  volumeClaimTemplates:
  - metadata:
      name: postgres-data
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: ebs-gp3-encrypted
      resources:
        requests:
          storage: 50Gi
```

4.6 Multi-AZ Implications for EBS

EBS volumes are AZ-specific. A volume created in ap-south-1a cannot be attached to a node in ap-south-1b. This has profound implications for EKS clusters spanning multiple AZs.

❑ **If you have a StatefulSet using EBS and the AZ where the EBS volume lives loses all nodes (e.g., a spot interruption wave), Kubernetes cannot reschedule the pod to another AZ — the volume is stuck in the dead AZ. This is a critical architectural consideration for HA.**

Mitigation strategies:

- Use WaitForFirstConsumer to ensure volumes are created in the same AZ where pods will run
- Use EFS instead of EBS for workloads that truly need multi-AZ resilience
- Run multiple replicas across AZs each with their own EBS volume (StatefulSet with N replicas, one per AZ)
- For databases: use a cloud-managed database (RDS) instead of self-managing on EBS for critical data

4.7 Interview Questions — Phase 4

Q: What is IRSA and why is it needed for the EBS CSI driver?

A: IRSA (IAM Roles for Service Accounts) allows Kubernetes service accounts to assume AWS IAM roles via OIDC federation. The EBS CSI driver needs to call AWS EC2 APIs (CreateVolume, AttachVolume, DeleteVolume) to manage EBS volumes. Without IRSA, the driver would need either node-level IAM permissions (overly permissive) or hardcoded access keys (insecure). IRSA gives the specific CSI driver service account precisely scoped IAM permissions via temporary credentials rotated automatically.

Q: Why is gp3 recommended over gp2 for new EBS volumes?

A: gp3 decouples IOPS and throughput from storage size. With gp2, IOPS scale at 3 IOPS/GB, so a 100GB volume gets 300 IOPS — you might need to over-provision storage just to get more IOPS. gp3 gives 3,000 IOPS baseline regardless of size, scalable independently up to 16,000 IOPS. gp3 is also approximately 20% cheaper than gp2 for the same capacity. There is almost no reason to use gp2 for new deployments.

Phase 5 — Real Production Scenarios

This phase covers what actually happens in production environments — failure scenarios, debugging workflows, and real-world architectural decisions.

5.1 StatefulSets vs Deployments for Stateful Workloads

Characteristic	StatefulSet vs Deployment
Pod identity	StatefulSet: stable, predictable pod names (postgres-0, postgres-1, postgres-2). Deployment: random hash suffixes (postgres-7d8f9c-xkj2p).
Ordered startup	StatefulSet: pods start in order (0, then 1, then 2). Next pod waits for previous to be Ready. Deployment: all pods can start simultaneously.
Ordered shutdown	StatefulSet: pods shut down in reverse order (2, then 1, then 0). Critical for Kafka — highest partition leader shuts down last.
Volume naming	StatefulSet: each pod gets its own PVC named data-postgres-0, data-postgres-1. Deployment: all pods share the same PVC (only possible with RWX).
Headless service	StatefulSet: uses headless service. Each pod gets a stable DNS: postgres-0.postgresql-headless.namespace.svc.cluster.local. Deployment: ClusterIP service load-balances across pods.
When to use	StatefulSet: databases, Kafka, Elasticsearch, Redis clusters. Deployment: stateless APIs, web servers, microservices without individual storage needs.

5.2 Volume Expansion — Online Disk Resize

allowVolumeExpansion: true in the StorageClass enables resizing a PVC without deleting it. The process for EBS volumes:

40. Edit the PVC to increase storage: `kubectl edit pvc postgres-data-postgres-0`
41. Change `spec.resources.requests.storage` from 50Gi to 100Gi
42. external-resizer detects the change and calls `ControllerExpandVolume` on the CSI driver
43. CSI driver calls AWS `ModifyVolume` — EBS volume is resized (this happens online, no downtime)
44. CSI driver calls `NodeExpandVolume` — the filesystem on the volume is resized (`resize2fs` for `ext4`)
45. The pod sees the increased storage without restart

```
# Expand PVC storage
```

```
kubectl patch pvc postgres-data-postgres-0 \
  --patch '{"spec":{"resources":{"requests":{"storage":"100Gi"}}}}' \
  -n production

# Watch the expansion
kubectl describe pvc postgres-data-postgres-0 -n production
# Look for: 'Normal FileSystemResizeSuccessful CSI volume expansion succeeded'
```

❑ You can only **INCREASE** volume size — you cannot shrink EBS volumes. Plan your initial size carefully. Also, volume expansion requires the pod to be running (for filesystem resize). If the pod is terminated, the volume resize will complete but filesystem resize waits for pod restart.

5.3 Volume Snapshots

VolumeSnapshotClass

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: ebs-vsc
  annotations:
    snapshot.storage.kubernetes.io/is-default-class: "true"
driver: ebs.csi.aws.com
deletionPolicy: Delete
```

Taking a Snapshot

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: postgres-snapshot-20260513
  namespace: production
spec:
  volumeSnapshotClassName: ebs-vsc
  source:
    persistentVolumeClaimName: postgres-data-postgres-0
```

Restoring from Snapshot

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data-restored
  namespace: production
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: ebs-gp3-encrypted
  resources:
    requests:
```

```
    storage: 50Gi
dataSource:
  name: postgres-snapshot-20260513
  kind: VolumeSnapshot
  apiGroup: snapshot.storage.k8s.io
```

5.4 Troubleshooting Commands — Production Debugging

Diagnosing a Pending PVC

```
# Check PVC status
kubectl get pvc -n production

# Detailed PVC events
kubectl describe pvc postgres-data-postgres-0 -n production
# Look for: ProvisioningFailed, no nodes available, quota exceeded

# Check StorageClass exists
kubectl get storageclass

# Check CSI driver pods
kubectl get pods -n kube-system | grep ebs-csi

# Check CSI controller logs
kubectl logs -n kube-system -l app=ebs-csi-controller \
  -c csi-provisioner --tail=100
```

Diagnosing a Pod Stuck in ContainerCreating

```
# Describe the pod
kubectl describe pod postgres-0 -n production
# Look for: 'Unable to attach volume', 'Timeout waiting for volume'

# Check VolumeAttachment objects
kubectl get volumeattachment
kubectl describe volumeattachment <name>

# Check external-attacher logs
kubectl logs -n kube-system -l app=ebs-csi-controller \
  -c csi-attacher --tail=100

# Check node events
kubectl get events -n production --sort-by='.lastTimestamp'
```

Common Production Failures

Failure	Root Cause and Resolution
PVC stuck Pending	No matching PV + no working provisioner. Check: CSI driver running? IAM permissions correct? OIDC provider registered? StorageClass exists?
Pod stuck ContainerCreating with volume timeout	EBS attach operation failed. Common causes: volume in wrong AZ, EC2 volume attachment limit reached, IAM permission missing <code>ec2:AttachVolume</code> .
EC2 attachment limit	Each EC2 instance type has a maximum number of EBS volumes it can have attached. <code>t3.medium</code> : 28 volumes. If you have many StatefulSets, you can hit this limit. Solution: use larger instance types or spread across more nodes.
Volume stuck in Terminating	Volume finalizer preventing deletion. Manually remove finalizer: <code>kubectl patch pv <name> -p '{"metadata":{"finalizers":null}}'</code> .
Database CrashLoopBackOff after reschedule	PGDATA directory conflict with EBS lost+found. Fix: set PGDATA to a subdirectory: <code>/var/lib/postgresql/data/pgdata</code> .
Slow storage performance	Wrong EBS type (<code>gp2</code> with small size = low IOPS). Solution: migrate to <code>gp3</code> and configure IOPS independently.

5.5 Interview Questions — Phase 5

Q: What is the difference between a StatefulSet and a Deployment for database workloads?

A: StatefulSets provide stable network identity (predictable pod names like `postgres-0`), stable storage (each pod gets its own PVC that survives pod rescheduling), and ordered startup/shutdown. Deployments have random pod names, share storage (only works with RWX volumes), and all pods start/stop simultaneously. For databases, StatefulSets are mandatory — each replica needs its own storage and predictable identity for replication protocols to work.

Q: A PostgreSQL pod keeps crashing after being rescheduled to a new node. Logs show filesystem errors. What could cause this?

A: Most likely the PGDATA directory conflict with EBS's lost+found directory. When EBS volumes are formatted for the first time, a lost+found directory is created at the root mount point. PostgreSQL requires an empty data directory on initialization. If `PGDATA=/var/lib/postgresql/data` (the default), PostgreSQL sees lost+found and refuses to initialize. Fix: set `PGDATA=/var/lib/postgresql/data/pgdata` to use a subdirectory within the EBS mount.

Phase 6 — Advanced Storage Concepts

6.1 Volume Topology and WaitForFirstConsumer

This is one of the most important concepts for multi-AZ EKS clusters and one of the most commonly misunderstood.

The problem: When a PVC is created, the default binding mode (Immediate) causes the PV (and therefore the EBS volume) to be created immediately — before any pod is scheduled. The CSI driver must choose an AZ arbitrarily. If it chooses ap-south-1a, but the scheduler later decides the pod should run on a node in ap-south-1b, the pod can never start — the EBS volume is in the wrong AZ.

The solution: volumeBindingMode: WaitForFirstConsumer. With this mode, PVC creation does NOT immediately trigger volume provisioning. The PVC stays Pending. When a pod referencing the PVC is scheduled, the scheduler picks a node first, then the CSI driver creates the EBS volume in the same AZ as that node. Volume and node are guaranteed to be co-located.

```
# Check if WaitForFirstConsumer is working
kubectl get pvc my-pvc -o jsonpath='{.status.phase}'
# Should show: Pending (until a pod is scheduled)

kubectl get pvc my-pvc -o yaml | grep selectedNode
# After pod scheduling, shows which node was selected
```

6.2 Volume Cloning

CSI supports volume cloning — creating a new PVC from an existing PVC as a data source. The new volume starts as an identical copy of the source.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-clone
  namespace: production
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: ebs-gp3-encrypted
  resources:
    requests:
      storage: 50Gi
  dataSource:
    name: postgres-data-postgres-0
    kind: PersistentVolumeClaim
```

6.3 Encryption and Security

EBS Encryption with KMS

EBS volumes can be encrypted at rest using AWS KMS. All data on the volume, all I/O, and all snapshots are encrypted. The CSI driver passes the KMS key ID in the CreateVolume API call.

```
parameters:
  encrypted: "true"
  kmsKeyId: arn:aws:kms:ap-south-1:051556733177:key/mrk-abc123
```

Security considerations:

- Use customer-managed KMS keys (CMK) not AWS-managed keys for compliance requirements
- Ensure the CSI driver's IAM role has kms:GenerateDataKeyWithoutPlaintext and kms:Decrypt permissions
- Enable EBS encryption by default at the account level: aws ec2 enable-ebs-encryption-by-default

6.4 Performance Tuning

filesystem selection — ext4 vs xfs

Filesystem	Characteristics
ext4 (default)	Mature, widely supported. Good for general workloads. Default in most Linux distributions. Suitable for databases up to medium scale.
xfs	Better performance for large files and high-throughput workloads. Preferred by Red Hat for enterprise databases. Use for PostgreSQL WAL volumes, Kafka log directories, Elasticsearch data. Set fsType: xfs in StorageClass parameters.

Mount Options for Database Workloads

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-gp3-db
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "16000"
  throughput: "1000"
  encrypted: "true"
mountOptions:
  - noatime          # Don't update access time on reads (significant performance
improvement)
  - nodiratime       # Same for directories
  - nobarrier        # Disable write barriers (safe with EBS which has its own
durability)
  - data=writeback   # ext4: writeback mode (faster writes, safe with EBS)
```

6.5 Interview Questions — Phase 6

Q: Explain WaitForFirstConsumer and why it matters in multi-AZ clusters.

A: WaitForFirstConsumer is a volumeBindingMode that delays PV creation until a pod referencing the PVC is scheduled. Without it (Immediate mode), the CSI driver creates an EBS volume in an arbitrary AZ. If the scheduler later places the pod on a node in a different AZ, the pod can never start — EBS volumes are AZ-specific and cannot be attached to nodes in different AZs.

WaitForFirstConsumer ensures the EBS volume is always created in the same AZ as the node that will use it.

Q: What filesystem would you recommend for a high-throughput Kafka deployment on EBS, and what mount options would you use?

A: XFS for Kafka. XFS handles large numbers of small sequential writes better than ext4 and is preferred for high-throughput logging applications. Mount options: noatime (avoid unnecessary access time updates on each log segment read), nodiratime, and nobarrier (EBS provides its own durability guarantees, so write barriers at the filesystem level add latency without benefit). Also use gp3 with IOPS provisioned to at least 3x the expected peak I/O, and throughput scaled to match Kafka's sequential write patterns.

Phase 7 — Practical Labs

Lab 1: Basic Dynamic Provisioning (Beginner)

Objective: Deploy a simple web server with persistent storage using dynamic EBS provisioning

```
# 1. Verify StorageClass exists
```

```
kubectl get storageclass
```

```
# 2. Create PVC
```

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

```
metadata:
```

```
  name: webserver-pvc
```

```
  namespace: default
```

```
spec:
```

```
  accessModes: [ReadWriteOnce]
```

```
  storageClassName: ebs-gp3-encrypted
```

```
  resources:
```

```
    requests:
```

```
      storage: 10Gi
```

```
kubectl apply -f pvc.yaml
```

```
kubectl get pvc webserver-pvc # Status: Pending (WaitForFirstConsumer)
```

```
# 3. Create Deployment
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: webserver
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: webserver
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: webserver
```

```
    spec:
```

```
      containers:
```

```
        - name: nginx
```

```
          image: nginx:alpine
```

```
          volumeMounts:
```

```
            - name: webdata
```

```
              mountPath: /usr/share/nginx/html
```

```
      volumes:
```

```
- name: webdata
  persistentVolumeClaim:
    claimName: webserver-pvc
```

```
kubectl apply -f deployment.yaml
kubectl get pvc webserver-pvc # Now: Bound
kubectl get pv # Auto-created PV
```

Validation: Write a file to the volume, delete the pod, verify the file persists when the pod restarts.

```
kubectl exec -it <pod-name> -- sh -c 'echo Hello > /usr/share/nginx/html/test.html'
kubectl delete pod <pod-name> # Pod is rescheduled
kubectl exec -it <new-pod-name> -- cat /usr/share/nginx/html/test.html
# Output: Hello - data persisted!
```

Lab 2: StatefulSet Database (Intermediate)

Objective: Deploy PostgreSQL as a StatefulSet with EBS storage, simulate failure, verify data survival

Use the complete StatefulSet YAML from Phase 4. After deployment:

```
# Verify StatefulSet and PVC
kubectl get statefulset postgresql -n production
kubectl get pvc -n production

# Connect to PostgreSQL and create test data
kubectl exec -it postgresql-0 -n production -- psql -U postgres -d tenantops
CREATE TABLE test_persistence (id SERIAL, message TEXT, created_at TIMESTAMP DEFAULT
NOW());
INSERT INTO test_persistence (message) VALUES ('This data must survive pod restart');
SELECT * FROM test_persistence;
\q

# Simulate pod failure
kubectl delete pod postgresql-0 -n production
kubectl get pod postgresql-0 -n production -w # Watch it restart

# Verify data survived
kubectl exec -it postgresql-0 -n production -- psql -U postgres -d tenantops \
  -c 'SELECT * FROM test_persistence;'
# Data should still be there
```

Lab 3: Volume Expansion (Advanced)

```
# Expand PVC from 50Gi to 100Gi
kubectl patch pvc postgres-data-postgres-0 -n production \
  --patch '{"spec":{"resources":{"requests":{"storage":"100Gi"}}}}'
```

```
# Watch expansion events
kubect1 describe pvc postgres-data-postgres-0 -n production | grep -A5 Conditions

# Verify inside the pod
kubect1 exec -it postgresql-0 -n production -- df -h /var/lib/postgresql/data
# Should show 100G total
```

Lab 4: Snapshot and Restore (Production-Grade)

```
# Create snapshot
kubect1 apply -f volumesnapshot.yaml
kubect1 get volumesnapshot -n production

# Verify snapshot is ready
kubect1 describe volumesnapshot postgres-snapshot-20260513 -n production
# Look for: ReadyToUse: true

# Create restored PVC
kubect1 apply -f restore-pvc.yaml

# Launch a pod using restored PVC to verify data
kubect1 run restore-verify \
  --image=postgres:15-alpine \
  --env PGDATA=/var/lib/postgresql/data/pgdata \
  --
overrides='{"spec":{"volumes":[{"name":"data","persistentVolumeClaim":{"claimName":"postgres-data-restored"}}],"containers":[{"name":"restore-verify","image":"postgres:15-alpine","volumeMounts":[{"name":"data","mountPath":"/var/lib/postgresql/data"}]}}]}'

# Query restored data
kubect1 exec -it restore-verify -- psql -U postgres -d tenantops \
  -c 'SELECT * FROM test_persistence;'
```

Phase 8 — Interview Mastery and Architecture Summary

8.1 Senior-Level Interview Questions

Q: Explain the complete flow from PVC creation to a pod reading from an EBS volume.

A: (1) PVC created → API server stores it as Pending. (2) external-provisioner detects PVC referencing `ebs.csi.aws.com` StorageClass. (3) With `WaitForFirstConsumer`, waits until pod is scheduled. (4) Scheduler places pod on node in `ap-south-1a`. (5) external-provisioner calls CSI `CreateVolume` → EBS volume created in `ap-south-1a`. (6) PV created in Kubernetes, PVC bound. (7) `AttachDetach` controller creates `VolumeAttachment`. (8) external-attacher calls `ControllerPublishVolume` → AWS `ec2:AttachVolume` called. (9) kubelet calls `NodeStageVolume` → formats with `ext4`, mounts to staging path. (10) kubelet calls `NodePublishVolume` → `bind-mounts` to pod's `mountPath`. (11) Pod starts, container reads/writes files.

Q: How would you design storage architecture for a Kafka cluster on EKS that must survive an AZ failure?

A: Run Kafka as a `StatefulSet` with 3+ brokers (one per AZ minimum). Each broker gets its own EBS `gp3` volume provisioned in its AZ via `WaitForFirstConsumer`. Topic replication factor of 3 with `min.insync.replicas=2`. Configure rack awareness using Kafka's `broker.rack` set to the AZ name (topology label). When one AZ fails: the two remaining brokers (in 2 other AZs) continue serving the partition leader role. Kubernetes does NOT reschedule the failed broker's pod to another AZ (EBS is AZ-bound), but since replicas exist in other AZs, the cluster remains functional. Recovery: when the AZ recovers, the pod restarts on the same node and the EBS volume reattaches.

Q: What causes 'volume node affinity conflict' error and how do you resolve it?

A: This error occurs when a pod is scheduled to a node that is in a different AZ than the EBS volume. Root cause: StorageClass using `Immediate` binding mode OR a previously bound volume in a different AZ. Resolution: (1) Delete the pod and PVC (if `Retain` policy, preserve data). (2) Switch StorageClass to `WaitForFirstConsumer`. (3) Recreate PVC — volume will be created in correct AZ. (4) To recover existing data: create a snapshot of the mis-AZ'd volume, restore the snapshot to a new PVC targeting the correct AZ.

Q: How does IRSA work for the EBS CSI driver and why is it more secure than node-level IAM?

A: IRSA uses OIDC federation. The EKS cluster has an OIDC provider. The CSI driver's ServiceAccount has an annotation with the IAM role ARN. When the CSI driver pod starts, the AWS SDK detects the projected service account token, calls the STS `AssumeRoleWithWebIdentity` API using that token, and receives temporary credentials scoped to the specific IAM role. Only the EBS CSI driver service account can assume the role — not the node, not other pods. Node-level IAM permissions would give ALL pods on the node EC2 permissions — a compromise of any pod could be leveraged to manipulate EBS volumes.

8.2 Architecture Decision Matrix

Requirement	Recommended Solution	Why
Single-AZ database (MySQL, PostgreSQL)	EBS gp3 with StatefulSet	Block storage, high IOPS, RWO, cost-effective
Multi-AZ shared filesystem	EFS with EFS CSI Driver	Managed NFS, RWX, cross-AZ, elastic capacity
High-performance ML training	EBS io2 or FSx for Lustre	Maximum IOPS, parallel reads, S3 integration
Shared web content, CMS	EFS or EBS with read replicas	RWX for EFS, or single-writer with read-only mounts
Kafka on EKS	EBS gp3, XFS, noatime mounts	Sequential writes, high throughput, per-broker isolation
Elasticsearch cluster	EBS gp3 per node	Independent storage per node, high IOPS for indexing
Redis with RDB persistence	EBS gp3 small size	Snapshot-based persistence, RWO, fast attach
etcd (Kubernetes internal)	Local SSD or io2	Maximum IOPS, minimum latency, data integrity critical

8.3 Production Best Practices Checklist

- ✓ Use gp3 for all new EBS volumes — not gp2
- ✓ Set volumeBindingMode: WaitForFirstConsumer for multi-AZ clusters
- ✓ Enable allowVolumeExpansion: true on all StorageClasses
- ✓ Use reclaimPolicy: Retain for production database volumes
- ✓ Enable EBS encryption with customer-managed KMS keys
- ✓ Set PGDATA to a subdirectory for PostgreSQL on EBS
- ✓ Use StatefulSets (not Deployments) for databases
- ✓ Monitor PVC usage — alert at 80% capacity
- ✓ Test volume snapshots and restores quarterly
- ✓ Use XFS filesystem for high-throughput sequential workloads (Kafka)
- ✓ Set noatime mount option for database and log volumes
- ✓ Keep CSI driver updated — follow driver release notes
- ✓ Implement Velero for cluster-wide backup including PV data

✓ Document your data recovery runbook and test it

8.4 Revision Summary

Concept	One-Line Definition
PV	The actual cloud storage resource (EBS volume) represented as a Kubernetes object
PVC	A storage request — binds to a matching PV
StorageClass	A template defining how to dynamically provision PVs
CSI Driver	The translator between Kubernetes storage requests and cloud storage APIs
external-provisioner	Sidecar that watches PVCs and calls CSI CreateVolume
external-attacher	Sidecar that watches VolumeAttachments and calls CSI ControllerPublishVolume
WaitForFirstConsumer	Delays volume creation until pod is scheduled, ensuring AZ co-location
IRSA	IAM Roles for Service Accounts — gives CSI driver scoped AWS permissions via OIDC
StatefulSet	Workload controller providing stable identity and individual storage per pod
ReclaimPolicy: Retain	Preserves PV and cloud volume after PVC deletion — for production databases
Volume Snapshot	Point-in-time copy of a PV — enables backup and restore workflows
Volume Expansion	Increasing PVC storage size without data loss or pod restart

Kubernetes Storage Mastery — From Ephemeral to Production-Grade

Understand why. Know how. Debug anything. Build with confidence.